

Question 1

Part 1 – Which hidden-technical-debt category explains each case?

(a) Entanglement (CACE). Within a trained model there are really no independent inputs, since all the weights are fit jointly. Once the rounding was altered, the delivery-time input carried a different distribution, and the following training run redistributed how much the model leaned on each signal. That is how a feature nobody had touched came out worse.

(b) Undeclared consumers. The dashboard team has built on top of the output table with no agreement in place, which leaves a live dependency the ML team cannot see. Renaming a column or rescaling the scores would quietly break their campaign, and there is no way for the ML team to find that out beforehand.

(c) Pipeline jungle (with glue code and configuration debt alongside it). Nothing records which script runs after which, and the parameters sit buried inside the scripts. Someone new cannot reproduce a training run, and there is no trail linking a given model back to the inputs and settings that produced it.

Part 2 – A fix for (c).

Rewrite the chain as stages in a `dvc.yaml`, each one declaring the inputs it consumes and the outputs it writes, and lift every hyperparameter out into a `params.yaml`. The execution order then comes from the file instead of from memory, and `dvc repro` skips any stage whose dependencies have not moved. Since the `.dvc` pointers are committed to Git, checking out a commit restores the matching data and configuration as well, not just the source. Adding an MLflow run around training would give the metric history that would have surfaced (a) as soon as the model was retrained.

Question 2 – Analysis

Best-performing run. Across the six `MLPClassifier` experiments, the best run used a learning rate of 0.001 with a batch size of 64. It achieved approximately 0.973 best validation accuracy and 0.975 test accuracy, while also giving the lowest final training loss of about 0.020 among the runs shown. This configuration therefore provided the strongest overall performance.

Overfitting. There is no clear evidence of overfitting over the 10 training epochs. Training loss kept decreasing while validation accuracy remained high and continued to improve, with the best validation performance occurring at the final epoch. If substantial overfitting were present, we would expect training loss to continue decreasing while validation accuracy deteriorated, which was not observed.

Which hyperparameter mattered more. Batch size had the larger observed effect on mean best validation accuracy. The mean accuracy decreased from 0.97290 at batch size 64 to 0.96740 at batch size 256, an effect of 0.00550. Changing the learning rate from 0.0005 to 0.001 produced a smaller effect of about 0.00437. Thus, batch size influenced validation performance slightly more than learning rate.

MLflow logging code added to the starter script

```
mlflow.log_param("learning_rate", learning_rate)
mlflow.log_param("batch_size", batch_size)
mlflow.log_param("hidden_layers", "(128,)")
mlflow.log_param("activation", "relu")
mlflow.log_param("optimizer", "adam")
mlflow.log_param("epochs", EPOCHS)
mlflow.log_param("seed", SEED)
mlflow.log_metric("train_loss", train_loss, step=epoch)
mlflow.log_metric("val_accuracy", val_accuracy, step=epoch)
mlflow.log_metric("best_val_accuracy", best_val_accuracy)
mlflow.log_metric("final_train_loss", train_loss)
mlflow.log_metric("test_accuracy", test_accuracy)
mlflow.log_metric("best_epoch", best_epoch)
```

The parameters are logged once at the start of each run, the two `step`-indexed metrics are logged inside the epoch loop so that MLflow stores a per-epoch curve, and the final four metrics are logged after training as summary metrics for the run.